

DATA Structure & Algorithms

- 1) (b) Data and a Pointer to the next node.
- 2) (c) Null
- 3) (b) Self-referential Structure
- 4) (b) $head == NULL$
- 5) (b) $newnode \rightarrow next = head; head = newnode;$
- 6) (b) $temp = head; head = head \rightarrow next; free(temp);$
- 7) (c) Traversing from tail to head.
- 8) (b) n
- 9) (c) Copying the data of $p \rightarrow next$ into p, then deleting $p \rightarrow next$
- 10) (b) $while (p != null)$
- 11) (c) $p == head$
- 12) (c) Two pointers advancing at diff. rates.
- 13) (c) Three
- 14) (b) Exactly one node
- 15) (b) $p = (struct node*) malloc (size of (struct node));$
- 16) (b) Avoid special - case handling for insertion and deletion
- 17) (c) The first node.

18) (b) It does not support random (indexed) access

19) (b) Loss of access to all nodes, causing a memory leak.

20) (b) A slow pointer and a first pointer.

21) (c) The last node.

22) (b) Nodes may be scattered anywhere in memory.

```
23) struct node {  
    int data;  
    struct node * next;  
};
```

24) The code prints the sum of each node and its next node:

$$\rightarrow 10 + 20 = 30$$

$$\rightarrow 20 + 30 = 50$$

$$\rightarrow 30 + 40 = 70$$

Output is $\rightarrow 30 \quad 50 \quad 70$

26) Error:- head is passing by value, so changes to the head pointer won't affect the original list. Also, an empty-list case needs handling.

Correct version:-

```
Void insertEnd (Struct node **head, int x) {
```

```
    Struct node *t = (Struct node*) malloc (size of (Struct node));
```

```
    t->data = x;
```

```
    t->next = NULL;
```

```
    if (*head == NULL) {
```

```
        *head = t;
```

```
        return;
```

```
    }
```

```
    Struct node *p = *head;
```

```
    while (p->next != NULL)
```

```
        p = p->next;
```

```
    p->next = t;
```

```
}
```



```

26) Struct node * reverse (Struct node * head) {
    Struct node * prev = NULL;
    Struct node * curr = head;
    Struct node * next = NULL;

    While (curr != NULL) {
        next = curr -> next;
        curr -> next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

27) Suppose, the list is 10 → 20 → 30 → 40 → NULL

⇒ I want to delete 30.

⇒ Then, first 20 node's next pointer

needs to be pointed to 40.

Before $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ NULL
Prev del next

After $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ $\dots$ $\rightarrow$ 40 $\rightarrow$ NULL
 $\uparrow$
30 deleted.

▣ prev $\rightarrow$ next = del $\rightarrow$ next;
free (del);

28) Initial list:-

5 $\rightarrow$ 8 $\rightarrow$ 12 $\rightarrow$ 3 $\rightarrow$ NULL

i) 7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 12 $\rightarrow$ 3 $\rightarrow$ NULL

ii) 7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$ NULL

iii) 7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$? $\rightarrow$ NULL

Final answer: 7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$? $\rightarrow$ NULL